

pv_geom: Scalable Extraction of Geometric Attributes for Solar Photovoltaic Installations from Airborne LiDAR

Working paper — draft v0.1

Job Taminiau

Foundation for Renewable Energy and Environment (FREE)

jt@freefutures.org

July 15, 2026

Abstract Existing large-area photovoltaic (PV) inventories resolve where solar installations are, but not how they are built. We present `pv_geom`, an open Python package that enriches a vector inventory of PV polygons with per-installation geometry derived from co-temporal classified airborne LiDAR: panel tilt and azimuth with bootstrap uncertainties, the orientation of the underlying roof plane, panel–roof angle, heights above ground and roof, and a rules-based mounting-type classification with per-row confidence and provenance. The pipeline is tile-centric and horizontally scalable: polygons are grouped by LiDAR tile, each tile group is processed independently on a Dask cluster (locally or on ephemeral cloud workers via Coiled), and results are written incrementally with per-group failure tolerance and crash-safe resume. Runs are reproducible bit-for-bit across backends via config hashing and per-polygon deterministic random seeds. We validate the method on metropolitan Phoenix, Arizona (~10 pts/m² USGS 3DEP LiDAR): on a 1,000-polygon utility-scale benchmark, all fits succeed with inlier RMSE of 1.3 cm (median), and local and cloud backends produce identical output; a 8,906-polygon mixed-density stress test completes in 25 minutes on 8 workers and demonstrates that the quality-flag system correctly isolates the long tail of small, sparse, or noisy input polygons rather than emitting unreliable fits. We discuss calibration and validation steps required before a metro-scale release.

i Note

Status. This is a working paper accompanying `pv_geom` v0.1.x. Benchmark results in Section 5 were produced with the six-label mounting taxonomy of v0.1.0; the software now ships a nine-label taxonomy (Section 3.5) whose new thresholds are uncalibrated defaults. The full ~417k-polygon Phoenix atlas run is pending and will be reported in a revision. Placeholders for those results are marked [TODO].

1 Introduction

Large-area PV inventories built from aerial or satellite imagery — e.g. DeepSolar for the United States [1], a harmonised open dataset for the UK [2], and a global inventory of utility-scale units [3] — resolve the *presence*, *extent*, and rough *capacity* of solar installations. What they do not resolve is *geometry*: the tilt and azimuth of the modules, whether they are flush-mounted on a pitched roof or racked above a flat one, whether a ground-mounted array is fixed or tracking, and how the panels relate to the structure beneath them.

Geometry matters for several downstream questions:

1. **Yield modeling.** Tilt- and azimuth-aware irradiance transposition is the difference between assuming every rooftop system faces south at latitude tilt and modeling the fleet that actually exists.
2. **Mounting-type classification.** Flush-mount residential, tilted commercial rack, carport, fixed ground mount, and single-axis tracker have different cost structures, capacity densities, and policy relevance.
3. **Equity and policy analysis.** Fused with parcel and demographic data, installation geometry sharpens questions about who adopts which kind of system — the level of detail currently available only from permit records such as Tracking the Sun [4], which cover a nonrandom subset of jurisdictions.
4. **Expansion headroom and shading.** Roof-plane orientation and panel-roof geometry are inputs to estimating remaining usable roof area.

Airborne LiDAR resolves this geometry directly. Where a LiDAR acquisition is roughly co-temporal with the imagery underlying a polygon inventory, the returns inside each PV polygon sample the panel surface itself, and the returns around it sample the roof or ground beneath. `pv_geom` operationalizes this observation at metropolitan scale: given a polygon inventory, a classified LAZ tile archive on S3, and a building-footprint layer, it produces a polygon-keyed attribute table with per-row quality flags and full provenance.

The package is deliberately metro-agnostic — all thresholds, coordinate systems, and point-classification conventions are configuration — and its first deployment target is metropolitan Phoenix, Arizona: a ~417,000-polygon PV inventory derived from a segmentation model applied to aerial imagery, paired with the USGS 3DEP Maricopa–Pinal 2020 LiDAR collection [5] at ~10 pts/m².

2 Data

PV polygons. A GeoParquet inventory of PV installation polygons with a unique `polygon_id`. For Phoenix we use an atlas of 416,976 polygons produced by a SAM-based segmentation pipeline over 2024 aerial orthoimagery (median polygon ~11 m²). MultiPolygons are exploded into single parts with a `parent_polygon_id` link. Polygons are reprojected once at ingest into the LiDAR CRS (Phoenix: EPSG:6341, NAD83(2011) / UTM 12N, metres).

LiDAR. Classified LAZ tiles with a sidecar tile index. The Phoenix archive (USGS LPC AZ MaricopaPinal 2020 B20) comprises 13,373 indexed 1-km tiles, of which 3,381 are currently mirrored to S3; delivered density is 9–11 pts/m². A practical complication: these tiles carry ASPRS classes 1 (unclassified), 2 (ground), and 7 only — there is no class 6 (“building”). `pv_geom` therefore supports a configurable fallback: when class 6 is absent, panel/roof candidate points are class-1 returns more than a configurable height (default 1.5 m) above the tile-group ground median.

Building footprints. FEMA USA Structures polygons for Arizona [6]. Roughly 92% of atlas polygons intersect at least one footprint; the remaining ~8% sit in genuine FEMA coverage gaps and surface in the

output as `on_building = False` (a data limitation we return to in Section 6). The FEMA `shape_area` attribute is unusable (all zeros in this extract); areas are always recomputed from geometry.

3 Methods

3.1 Panel-plane fitting

For each polygon, panel-class returns are clipped to the polygon eroded inward by 0.15 m (suppressing racking-edge and roof returns at the boundary), and a plane is fit by RANSAC [7] with an inlier distance threshold of 0.05 m and 200 iterations, followed by a least-squares refinement (PCA: the plane normal is the smallest-variance right singular vector of the centered inlier cloud), re-classification of inliers against the refined plane, and RMSE computation on the final inlier set.

With unit normal $\hat{n} = (n_x, n_y, n_z)$ canonicalized to the upper half-space ($n_z \geq 0$), tilt and compass azimuth follow as

$$\theta_{\text{tilt}} = \arccos(n_z), \quad \phi_{\text{az}} = \text{atan2}(n_x, n_y) \bmod 360^\circ,$$

with azimuth reported as NaN below a 1° tilt floor, where the direction a near-horizontal panel “faces” is numerically and physically ill-defined.

A fit is attempted whenever at least 3 points survive clipping, but rows are flagged `low_density` below 3 pts/m² or 30 points, and `poor_fit` when the RANSAC consensus covers less than 60% of clipped points. The 30-point floor is empirical: a bootstrap size-sweep on clean Phoenix panels showed tilt estimates are *unbiased* down to $N=5$ with precision scaling as $1/\sqrt{N}$, so the practical floor is not statistical noise but RANSAC’s ability to reject overshoot and multi-plane contamination, which requires roughly 30 inliers. At Phoenix densities that corresponds to $\sim 3 \text{ m}^2$ — about the bottom 10–15% of atlas polygons.

Uncertainty. $1\text{-}\sigma$ uncertainties for tilt and azimuth are estimated by a 50-sample nonparametric bootstrap over the inlier set, refitting each resample by PCA. Azimuth dispersion uses the Mardia circular standard deviation $\sigma = \sqrt{-2 \ln R}$, where R is the mean resultant length [8], avoiding the wrap-around artifacts of a naive standard deviation on angles.

3.2 Multi-plane detection

East–west racking on flat commercial roofs produces two panel populations facing $\sim 180^\circ$ apart. After the primary fit, a second RANSAC pass runs on the primary’s non-inliers (looser 0.10 m threshold); if the secondary plane captures at least 20% of the polygon’s points, both planes are reported, and the row is flagged `east_west_rack` when the two azimuths are within 25° of opposite and tilts agree within 5° .

3.3 Roof-plane extraction

To characterize the surface *beneath* a rooftop system, `pv_geom` fits a plane to panel-class returns in a ring buffer around the polygon: the polygon buffered outward (starting at 3 m), minus the polygon itself, minus any other PV polygons on the same building, intersected with the chosen building footprint (the intersecting footprint with maximum overlap). If fewer than 100 returns fall in the ring, the buffer expands in 0.5 m steps to a 5 m cap before giving up with flag `roof_insufficient` — the common failure mode when panels occupy most of the roof. A fit whose inlier RMSE exceeds 0.10 m is rejected as `roof_complex` (dormers, hips, multi-facet bleed). From the panel and roof fits together we derive the panel–roof angle (angle between the two normals) and height-above-roof (vertical offset of the panel inlier centroid from the roof plane).

3.4 Heights

Height-above-ground is the median panel-inlier elevation minus the median class-2 (ground) elevation within 10 m of the polygon centroid. When the class-6 fallback is active (Section 2), the ground reference for the *candidate filter* is a single tile-group-wide ground median — defensible on Phoenix’s flat terrain at the 1-km tile scale, and revisit-worthy in higher-relief metros.

3.5 Mounting classification

Mounting type is assigned by an ordered rules engine over the features (`on_building`, `panel_tilt`, `panel_roof_angle`, `height_above_roof`, `height_above_ground`, `area`, `aspect_ratio`, `roof_plane_available`, `roof_tilt`, `east_west_rack`). Rules are evaluated most-specific-first; the first rule to clear a confidence of 0.5 fires. Each threshold comparison maps to a confidence via a piecewise-linear ramp — 0.5 exactly at the threshold, 1.0 at a fractional margin (default 50% of the threshold) past it — and a rule’s confidence is the minimum over its clauses. Polygons matching no rule are *ambiguous* with confidence 1 — $\max(\text{near-miss confidences})$.

Table 1: Mounting rules in evaluation order. Rule IDs are stable identifiers, not evaluation order: R7 and R8 were added after the initial six and run before the broader rules that would otherwise absorb their polygons.

Rule	Label	Core predicate
R7	<code>east_west_rack_rooftop</code>	on building, EW two-plane signature, tilt $\leq 20^\circ$; if a roof fit exists it must be flat (guards against gable facets, which also face $\sim 180^\circ$ apart)
R1	<code>flush_mount_pitched_roof</code>	on building, panel–roof angle $\leq 5^\circ$, height above roof ≤ 0.5 m; label splits on roof tilt vs 7°
R2	<code>tilted_rack_rooftop</code>	on building, panel–roof angle $> 5^\circ$ and height above roof ≤ 1.5 m (or, without a roof fit: tilt $> 5^\circ$ and height above ground > 2.5 m)
R8	<code>pole_mount</code>	off building, height ≥ 2 m, area $\leq 20 \text{ m}^2$, aspect ≤ 2
R3	<code>carport</code>	off building, height ≥ 2 m, aspect ≥ 2
R4	<code>ground_mount_tracker_suspected</code>	off building, height < 2 m, aspect ≥ 4 , tilt $< 35^\circ$
R5	<code>ground_mount_fixed</code>	off building, height < 2 m, tilt $\geq 5^\circ$
R6	<code>ambiguous</code>	default

Thresholds are configuration, not code; the R7/R8 and flat/pitched-split defaults are physically motivated but *uncalibrated* pending a labeled validation sample (Section 6). The classifier sits behind an abstract interface so a learned model can replace the rules without touching the pipeline; rules would then remain as a fallback and explanation layer.

3.6 Quality flags and provenance

Every row carries its quality flags (`low_density`, `poor_fit`, `near_horizontal`, `east_west_rack`, `tracker_suspected`, `roof_insufficient`, `roof_complex`), the contributing LiDAR tile IDs, the package version, a SHA-256 hash of the fully-resolved configuration, and a run ID. A JSON manifest per run records the resolved config, input URIs, cluster spec, counts, and aggregate quality statistics. Reproducibility is

by construction: RANSAC and bootstrap seeds derive from a SHA-256 of the polygon ID (process-stable, unlike Python’s salted `hash()`), so identical inputs and config produce identical output regardless of worker count, scheduling order, or compute backend.

4 Implementation and scaling

LAZ I/O dominates cost, so the pipeline is *tile-centric*: each polygon is assigned a primary tile (containing its centroid) plus the set of tiles its geometry overlaps; polygons are grouped by primary tile; and each tile group’s fetch list expands to the union of overlapping tiles. One Dask task [9] processes one tile group: fetch all tiles (with local caching), split the point cloud by class once for the whole group, then clip and fit per polygon. Only the primary group emits a polygon’s row, so tile-spanning polygons need no dedup pass and raw point clouds are never shuffled between workers.

Three design decisions matter at atlas scale (~1,000–2,000 tile groups):

- **Incremental writes with per-group failure tolerance.** Each partition is written the moment its group completes; a failed group is recorded in the manifest rather than aborting the run, and `--resume` retries exactly the missing partitions. A single worker loss at hour 3 of a multi-hour run costs one group, not the run.
- **Slim task graphs.** Tasks embed only their group’s tile URIs (not the 13k-entry index), and a single client-side S3 LIST drops indexed tiles with no LAZ object (~75% of the Phoenix index) before dispatch, replacing thousands of worker-side 404 round-trips.
- **Bounded memory.** Class filtering is hoisted out of the per-polygon loop and the raw 4-column point array (~1.5 GB on dense tiles) is freed before fitting begins. Phoenix’s densest tiles (~3,000 polygons, >50M points) still require 16 GiB workers.

The same code runs serially in-process (`--no-dask`), on a local `distributed.LocalCluster`, or on ephemeral cloud workers provisioned by Coiled in the data’s AWS region, selected purely by configuration.

5 Validation and benchmarks

5.1 Point-level spike validation

A pre-implementation spike on 29 randomly drawn Phoenix polygons established that the data supports the method: 27/29 plane fits achieved $RMSE < 0.05$ m; the tilt distribution was bimodal at 5–10° (flush-mount) and 18–22° (tilted rack); and azimuths concentrated at 150–230° (south to SSW) with none facing north — physically correct for Arizona. On one polygon with visible segmentation overshoot into a yard, RANSAC reduced RMSE from 0.377 m (plain SVD fit) to 0.018 m by rejecting the 14% of returns off the panel plane, confirmed by aerial overlay.

5.2 1,000-polygon benchmark (utility-scale cluster)

A 1 km² bounding box capturing a utility-scale plant (Figure 1): 1,000 polygons across 2 tile groups, all fits successful, panel RMSE p50/p90 = 1.3/1.6 cm. Wall time was 322 s on a 2-worker local cluster and 414 s on Coiled (the difference is ~90 s cluster spin-up, ~25 s worker package install, and cold S3 reads), with **bit-for-bit identical** algorithm output across backends. The classifier splits the plant into 617 `ground_mount_tracker_suspected` and 363 `ground_mount_fixed` — a known weakness of the per-polygon tracker heuristic that installation-level clustering is designed to repair (Section 6).

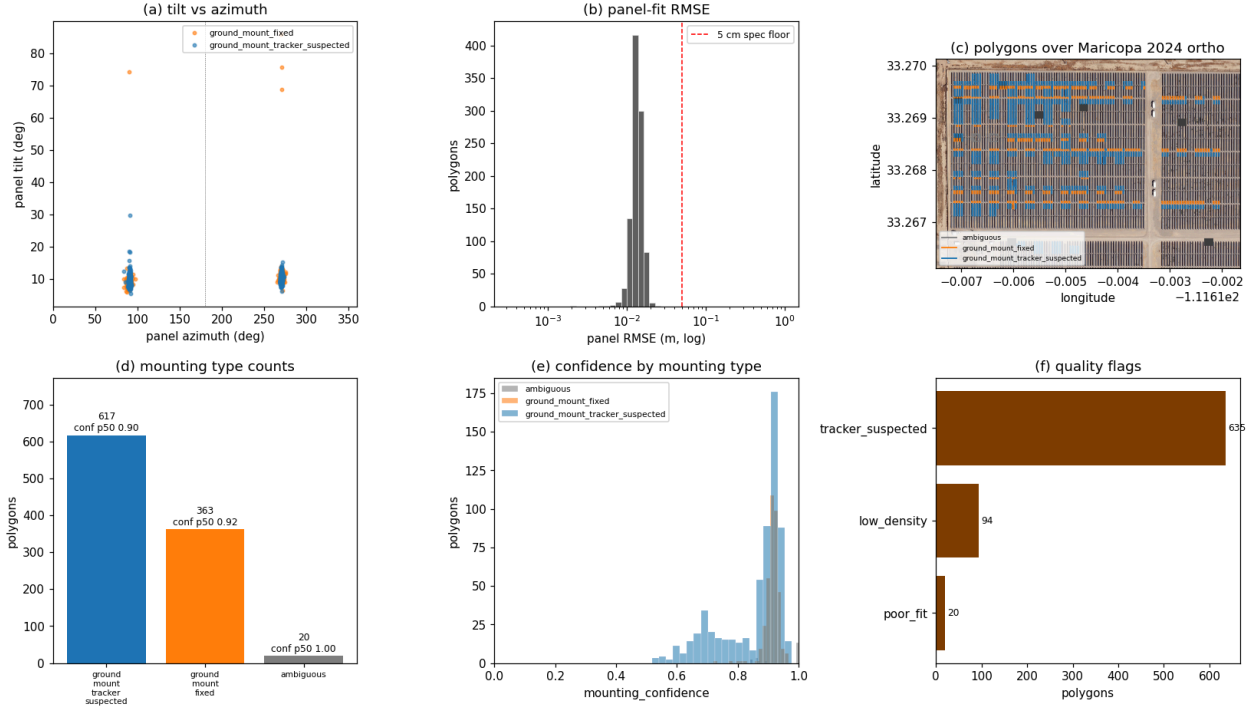


Figure 1: Six-panel review of the 1,000-polygon benchmark: (a) tilt vs azimuth, (b) panel-fit RMSE against the 5 cm specification floor, (c) polygons over 2024 orthoimagery, (d) mounting-type counts, (e) confidence by mounting type, (f) quality flags.

5.3 8,906-polygon stress test (mixed density)

A 3 km × 3 km box spanning 8 tiles and the full quality range of the input atlas, run on 8 Coiled workers (2 vCPU / 16 GiB): 24.8 min wall, all 8,906 rows emitted. Here the input long tail becomes visible: 68% of panel fits succeed; 42% of rows carry `low_density`, 32% `poor_fit`, and 33% classify as `ambiguous`. Critically, *successful* fits remain excellent (RMSE p50/p90 = 1.5/1.8 cm): the flag system is correctly refusing the small, sparse, and segmentation-noisy polygons rather than emitting junk geometry on them. We treat the flagged share as an honest property of the input inventory, to be addressed by input-side repair (fragment clustering, adaptive erosion) rather than by loosening fit acceptance.

5.4 Full-atlas run

[TODO] The full Phoenix atlas run (~417k polygons, of which ~25% fall on currently-mirrored LiDAR tiles → ~100k processable; projected 2–4 h on 40 workers) is the next gating item. This section will report end-to-end counts, mounting-type composition, tilt/azimuth distributions by mounting type, and cost.

6 Limitations and planned work

1. **Footprint gaps.** ~8% of polygons are off-building solely because FEMA lacks the structure, shunting true rooftop systems toward ground/carport/ ambiguous rules. Planned: fall back to a second footprint source where FEMA is empty.
2. **Uncalibrated classifier thresholds.** All rule thresholds — especially the new R7/R8 and flat/pitched defaults — await calibration against a ~200-polygon aerial-verified label set, which will also resolve whether the 33% ambiguous rate in the stress test reflects miscalibration or genuinely uninformative inputs.

3. **Per-polygon tracker heuristic.** Trackers are detected per polygon (elongated, low, off-building, modest tilt); the 1k benchmark shows one plant split across two labels. Planned: installation-level clustering (grouping by centroid proximity and orientation coherence), which also enables site typing, row-spacing/GCR estimates, and repair of segmentation fragments by inheritance from the cluster fit.
4. **Fixed erosion.** The 0.15 m inward erosion consumes a large share of points on the smallest polygons and drives part of the `low_density` rate; erosion should scale with polygon size.
5. **External validation.** Tilt/azimuth/mounting accuracy will be quantified against permit and interconnection records (e.g., Tracking the Sun [4]) for a Phoenix sample.
6. **Flat-terrain assumption in the class-6 fallback.** The tile-wide ground median used to select above-ground candidate returns is appropriate for Phoenix but should become slope-aware before deployment in high-relief metros.

7 Software availability

`pv_geom` is a Python package (src layout, [pyproject.toml](https://github.com/JobTaminiau/pv_geom/blob/main/pyproject.toml)) with a Typer CLI, Pydantic-validated YAML configuration, 152 unit tests on synthetic inputs, and a gated integration test against real Phoenix data. Source: https://github.com/JobTaminiau/pv_geom. A tagged release with an archived DOI (Zenodo) and license is in preparation; until then the repository URL is the citable reference. LiDAR: USGS 3DEP / LPC [5], hosted under the ASU NSF Phoenix collection. Building footprints: FEMA USA Structures [6].

Acknowledgments

Built at the Foundation for Renewable Energy and Environment (FREE).

References

Bibliography

- [1] J. Yu, Z. Wang, A. Majumdar, and R. Rajagopal, “DeepSolar: A Machine Learning Framework to Efficiently Construct a Solar Deployment Database in the United States,” *Joule*, vol. 2, no. 12, pp. 2605–2617, 2018, doi: 10.1016/j.joule.2018.11.021.
- [2] D. Stowell *et al.*, “A harmonised, high-coverage, open dataset of solar photovoltaic installations in the UK,” *Scientific Data*, vol. 7, p. 394, 2020, doi: 10.1038/s41597-020-00739-0.
- [3] L. Kruitwagen, K. T. Story, J. Friedrich, L. Byers, S. Skillman, and C. Hepburn, “A global inventory of photovoltaic solar energy generating units,” *Nature*, vol. 598, pp. 604–610, 2021, doi: 10.1038/s41586-021-03957-7.
- [4] G. Barbose, N. Darghouth, E. O’Shaughnessy, and S. Forrester, “Tracking the Sun: Pricing and Design Trends for Distributed Photovoltaic Systems in the United States,” technical report, 2023. [Online]. Available: <https://emp.lbl.gov/tracking-the-sun>
- [5] U.S. Geological Survey, “3D Elevation Program (3DEP) Lidar Point Cloud.” 2020.
- [6] Federal Emergency Management Agency, “USA Structures.” 2022.
- [7] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, doi: 10.1145/358669.358692.

- [8] K. V. Mardia and P. E. Jupp, *Directional Statistics*. Chichester: John Wiley & Sons, 2000.
- [9] M. Rocklin, “Dask: Parallel Computation with Blocked algorithms and Task Scheduling,” in *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, 2015, pp. 126–132. doi: 10.25080/Majora-7b98e3ed-013.